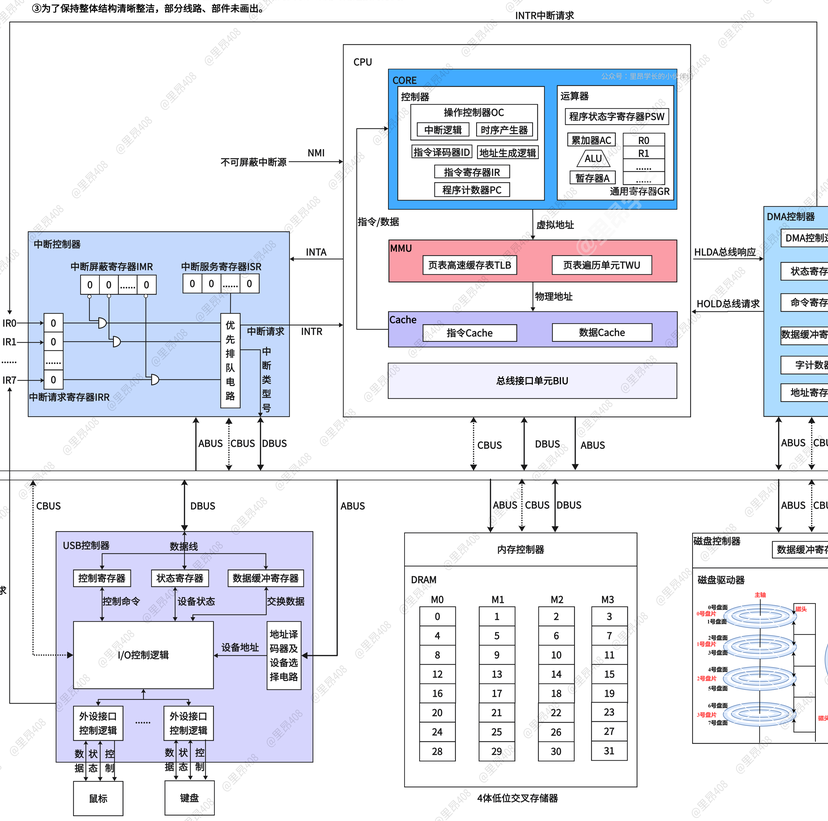
- CPU
  - CPU的基本构造
    - 核心CORE
      - 控制器 — 控制器里都有什么？控制信号是如何发出的？
        - 硬布线
        - 微程序
      - 运算器
        - ALU是如何工作的？
          - 逻辑电路与加减法运算
          - 整数加减法运算器
          - 标志寄存器
        - 乘除法如何实现
          - ALU+移位计算
          - 阵列乘法器/除法器
        - 寄存器里有什么？ — 如何用机器数表示真值？
          - 定点数：原、反、补、移、无符号数
          - 浮点数：IEEE754
    - 内存管理单元MMU
      - 页表高速缓存表（Table Lookup Buffer，简称TLB） — 当TLB中缓存着要访问的虚拟地址到物理地址的转换关系的时候，就可以直接找到的物理地址，通过该物理地址在cache中查找数据。
      - 页表遍历单元（Table Walk Unit，简称TWU） — 如果TLB中没有缓存要访问虚拟地址对应的物理地址，这时会通过TWU模块遍历主存中的页表，查找相应的物理地址，找到后再通过这个物理地址访问cache。
    - 高速缓冲存储器Cache
      - 若cache中存在对应数据，则无需访问主存，否则，还要进行cache缺失处理。
      - 映射方式
    - 总线接口单元BIU — 总线控制器负责总线控制权的仲裁以及总线资源的分配和管理，所有设备都必须通过总线接口与总线连接，这里 DRAM 通过内存控制器、I/O设备通过 I/O 接口与总线相连。
  - CPU的工作原理
    - 存储程序工作方式 — 不断取出指令并执行
      - 指令是如何构造而成的？
        - 指令的基本格式
        - 指令的寻址方式
      - 都有哪些常用指令？ — X86、MIPS架构常用指令 — 两者的区别是什么？ — CISC和RISC
      - 指令执行的过程是什么样的？
        - 分为哪几个阶段？ — 取指、间指、执行、中断
        - 如何精准控制？ — 时序系统 — 现代计算机的时钟信号
        - 数据是如何流动的？
          - 数据通路
            - 数据通路的基本结构？
            - 数据通路是如何构成的？
          - 如何描述数据流动的过程 — 微操作
      - 嫌CPU执行的慢怎么办 — 指令流水线
  - 如何衡量一个CPU的好坏 — CPU的性能指标 — 平均CPI、字长

注：①本结构图与真实的计组结构有所出入，旨在从考研人的角度，带大家把各个计算机部件联系起来。
②部分信号可能是通过总线传输，但为了强调不同部件之间的关系，采用专用连接的方式。
③为了保持整体结构清晰整洁，部分线路、部件未画出。



- DMAC
  - DMAC的基本构造（不唯一）
    - 地址计数器 — 它用于存放交换数据的内存地址。在DMA预处理阶段由CPU设置，每次 DMA 传送后自动递增，指向下一个内存单元。
    - 字计数器 — 它用于记录数据块的长度，也在预处理阶段设置。传送开始后，每传送1个字或字节就自动减1。当计数值为0时，数据传送完毕，发出DMA中断请求信号。
    - 数据缓冲寄存器(DBR) — 用来暂存每次传送的数据。输入时，数据由外设(如磁盘)先送往数据缓冲寄存器，再通过数据总线送到主存。反之，输出时，数据由主存通过数据总线送到数据缓冲寄存器，然后再送到外设。当然，若不经过该寄存器直接通过数据总线实现设备和内存之间的数据交换，则性能更好。
    - 命令寄存器（DCR） — 它用于接收CPU的控制命令。
    - 状态寄存器（DSR） — 它负责向 CPU 反馈 DMA 控制器状态。
    - DMA控制逻辑 — 它包括控制和时序电路，以及状态标志，用来修改内存地址计数器和字计数器，指定数据传输方向，并对 DMA 请求信号和CPU响应信号进行协调和同步；控制逻辑还应该支持中断机制，一组数据交换完毕时，向CPU发出DMA中断请求，通知CPU进行结束处理。
  - DMA工作原理
    - 初始阶段、传输阶段、结束阶段
    - CPU、DAMC、设备
  - DMA信号
    - DREQ信号 — 数据传输阶段，设备准备好数据后就通过 DREQ 控制线向 DMAC 发出 DMA 请求。
    - HOLD信号/HRQ请求信号 — DMAC需要申请总线控制权时，会通过系统总线中的HOLD信号向CPU发出总线使用请求。
    - HLDA信号 — CPU 中的总线控制器如果允许，可以向 DMAC 发出总线授权信号HLDA。
    - DACK信号
      - DMAC收到HLDA信号即获得总线控制权，此时DMAC将内存地址放置在地址总线上
        - 如果是读命令，则将内存写和I/O读的控制信号送入控制总线，并向设备发出 DMA应答信号 DACK。设备收到 DACK 信号后会将一个机器字的数据放置在数据总线上，在内存写信号的控制下，数据最终会写入内存中，从而完成一次数据交换。
        - 如果是写命令，传输方向和控制信号正好相反。
      - 注意这里数据是直接通过总线交换的，并没有通过DMAC进行中转；**当然也可以通过DMAC 中的数据缓冲寄存器进行中转**，该方式相对而言性能会差一些。
  - DMAC的地址线 — 在DMA方式下，地址线呈输出状态，可对主存进行地址选择；在CPU方式下，地址线呈输入状态，可对DMA控制器中的有关寄存器进行寻址。

- 总线
  - 总线的基本构造
    - 一组连接各功能部件的传输线缆
    - 总线控制器
    - 总线接口
  - 总线的性能指标 — 总线带宽
    - 总线工作频率
    - 总线宽度
  - 总线事务 — 传送地址和命令、存储器准备数据、传送数据
  - 总线分类
    - 数据总线 — 双向 — 数据线用来承载在源部件和目的部件之间传输的信息，这个信息可能是CPU和主存之间交换的数据或从主存取出的指令，也可能是CPU向I/O模块发出的控制命令或从I/O模块收集到的状态信息。如果数据线和地址线复用的话，数据线上也可以传送地址信息。
    - 地址总线 — 单向 — 不考虑DMAC等特殊设备，一般默认地址线用来给出源数据或目的数据所在的主存单元或I/O模块的地址。地址线是单向的，总是由CPU将地址信息送到地址线，随后传送给CPU要访问的主存储器或I/O模块。
    - 控制总线 — 控制总线用于传输控制信号和时序信号。控制总线的传送方向由具体控制信号决定，既有 CPU送往功能部件的读写控制信号，也有功能部件发送给CPU的状态信号、请求信号等。它对某一根信号线来说是单向传输的，而对整体来说又是双向传输的。控制总线的位数要根据系统的实际需要而定。
      - 单向？双向？
      - 常见的控制信号和时序信号
        - 存储器读 — 将主存指定单元的数据读取到数据总线。
        - 存储器写 — 将数据总线上的数据写入地址总线指定的主存单元。
        - I/O 读 — 将地址总线指定的 I/O 端口的数据读取到数据线。
        - I/O 写 — 将数据总线上的数据写入地址总线指定的 I/O 端口。
        - 传输应答 — 表示数据已被接收或已传输到总线上。
        - 总线请求 — 表示某部件需要获得总线控制权。
        - 总线许可 — 表示申请总线控制权的部件已获得了控制权。
        - 总线正忙 — 表示当前总线正在使用。
        - 中断请求 — 表示某部件产生了中断请求。
        - 中断响应 — 表示中断请求已被接收。
        - 时钟信号 — 用于同步各种操作。
        - 复位信号 — 用于初始化各模块。

- 磁盘
  - 基本构造
    - 磁盘控制器
    - 磁盘驱动器
      - 里面都有什么？
        - 硬盘盘片
        - 主轴
        - 传动手臂
        - 磁头
      - 是如何工作的？
      - 磁盘的地址结构 — 柱面号、盘面号、扇区号
  - 如何衡量一个磁盘的好坏？ — 磁盘存储器的性能指标 — 存取时间
    - 寻道
    - 旋转延迟
    - 传输
  - 一个磁盘不够稳、不够快怎么办 — 磁盘阵列
    - RAID0
    - RAID1
    - ……
  - 磁盘性能还是不够怎么办 — 固态硬盘

- 中断控制器
  - 中断控制器的基本构造
    - 中断请求寄存器（IRR） — 由于每个外部设备中断源发出中断请求的时间是随机的，请求从发出到响应可能需要经过一定的时间，因此需要将不同外部设备的中断请求暂存在 IRR 中。
    - 中断屏蔽寄存器（IMR） — 倘若某个中断源希望在中断处理的过程中屏蔽掉部分中断源，则可以通过将其中断屏蔽位置1来屏蔽。
    - 优先级排队电路（优先编码器） — 多个中断请求可通过优先级排队电路进行排队，输入为多个中断请求信号，输出为多个请求中最高优先级的中断编号（类型号）和中断请求信号。
    - 中断服务寄存器（ISR） — 中断服务寄存器在多重中断中用于存放正在被服务的中断请求，包括已经得到中断响应但中断服务未结束的所有中断请求，每一位对应一个设备
  - 中断响应优先级 — 响应优先级、处理优先级
    - ① 不可屏蔽中断>内部异常>可屏蔽中断。
    - ② 内部异常中，硬件终止属于最高级，其次是指令异常或自陷等程序故障。
    - ③ DMA 中断请求优先于 I/O 设备传送的中断请求。
    - ④ 在 IO 传送类中断请求中，高速设备优先于低速设备，输入设备优先于输出设备，实时控制设备优先于普通设备。
  - 中断处理的整个过程
    - 中断识别 — 中断向量、向量地址、中断类型号
    - 中断响应 — 关中断、保持断点和程序状态字、送中断向量
    - 中断处理程序 — 注意单重中断和多重中断的区别

- I/O接口
  - I/O接口的基本构造
    - 数据缓冲寄存器 — 暂存外设与 CPU 或内存之间传送的数据信息
    - 状态寄存器 — 记录接口和设备的状态信息
    - 控制寄存器 — 保存 CPU 对外设的控制信息
    - 地址译码器及设备选择电路 — 用于识别地址总线上的地址是否是当前 I/O接口连接的外部设备。
    - I/O 控制逻辑
      - ① 将控制寄存器中的命令字译码成控制信号送到外设
      - ② 在数据缓冲寄存器和外设之间发送或接收数据
      - ③ 将外设状态收集到状态寄存器中
  - I/O接口如何编址
  - 工作原理
    - 程序查询方式 — 独占查询 / 定时查询
    - 程序中断方式

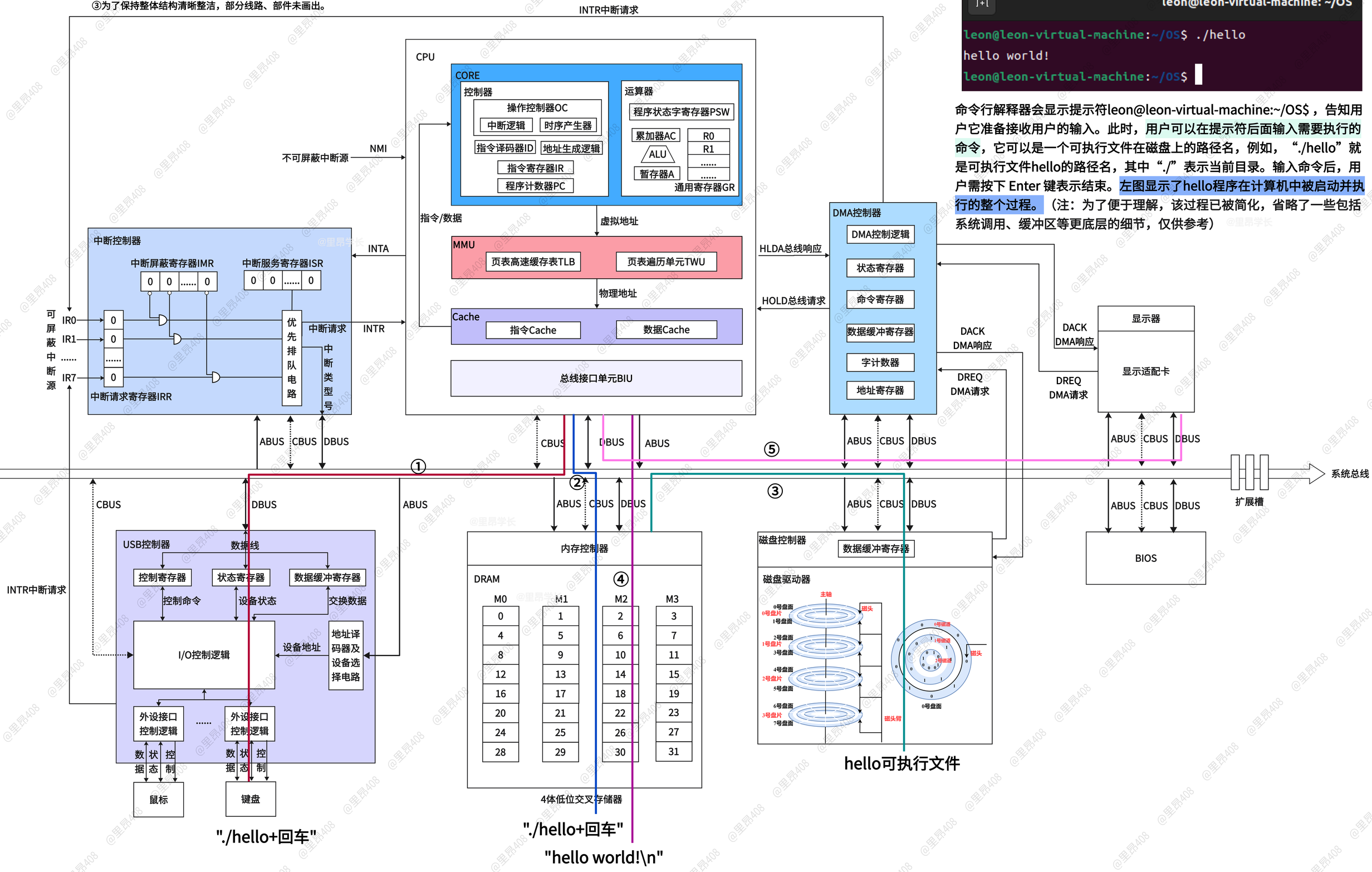
- 内存
  - 内存的基本构造
    - 内存控制器
    - DRAM芯片
      - DRAM芯片的结构、工作原理？
        - 行列地址复用
        - 行缓冲
      - DRAM刷新？
  - 主存容量不够大怎么办？ — 字扩展、位扩展
  - 主存速度不够快怎么办？ — 多模块存储器 — 低位交叉存储器
    - 如何计算带宽
    - 同时启动/轮流启动
  - 如何衡量一个主存的好坏？ — 与存储器相关的性能指标
    - 存储周期
    - 存储带宽 — 单位时间内存储器准备好的数据量
  - CPU是如何访存的？
    - 突发方式
    - 非突发方式
  - 虚拟页式存储器（OS）

对于一个存放在磁盘上的可执行文件，可以在操作系统提供的用户操作环境中，采用双击对应图标或在命令行中输入可执行文件名等多种方式来启动执行。例如，对于可执行文件 hello，通过命令行解释器启动执行的结果如下。

```
leon@leon-virtual-machine: ~/OS
leon@leon-virtual-machine:~/OS$ ./hello
hello world!
leon@leon-virtual-machine:~/OS$
```

命令行解释器会显示提示符leon@leon-virtual-machine:~/OS$ ，告知用户它准备接收用户的输入。此时，用户可以在提示符后面输入需要执行的命令，它可以是一个可执行文件在磁盘上的路径名，例如，“./hello”就是可执行文件hello的路径名，其中“./”表示当前目录。输入命令后，用户需按下 Enter 键表示结束。左图显示了hello程序在计算机中被启动并执行的整个过程。（注：为了便于理解，该过程已被简化，省略了一些包括系统调用、缓冲区等更底层的细节，仅供参考）

注：①本结构图与真实的计组结构有所出入，旨在从考研人的角度，带大家把各个计算机部件联系起来。
②部分信号可能是通过总线传输，但为了强调不同部件之间的关系，采用专用连接的方式。
③为了保持整体结构清晰整洁，部分线路、部件未画出。



hello程序在计算机中被启动并执行的整个过程

1. 假设采用程序中断I/O方式，用户在键盘输入字符后，首先会被保存到I/O接口中的数据缓冲寄存器中，随后I/O接口发出中断请求，CPU响应中断后，会将字符从数据缓冲寄存器读入到CPU内部的寄存器中(对应线①)，然后再保存到主存储器中（对应线②）。
2. 命令解释程序会扫描主存中的字符串，当读取到回车键对应的字符编码时，它就知道用户已经完成了当前命令的输入。之后，命令解释程序会对命令进行解析，并调用操作系统内核的相关服务例程来执行该命令，加载磁盘上的可执行文件 hello 到存储器中(对应线③)。
3. 内核加载完可执行文件中的代码及其所要处理的数据("hello world!\n")后，将hello 程序第一条指令的地址送到程序计数器(PC)中。
4. CPU随后开始执行hello程序，hello程序申请内核的相关服务例程，将主存的字符串"hello world!\n"从主存取到CPU的寄存器中(对应线④)，然后将 CPU寄存器中的字符送到显示器上显示出来(对应线⑤)。

从上述过程可以看出

一个程序被启动执行，必须依靠操作系统的支持，包括外壳程序和内核服务例程。例如，命令行解释器是操作系统外壳程序，用来对用户从键盘输入的命令进行解释，并调出操作系统内核来加载用户程序。

程序的执行过程就是数据在CPU、主存储器和I/O模块之间流动的过程，所有数据的流动都通过总线等进行。数据在总线上传输之前，需要先缓存在存储部件中，因此，除了主存储器本身是存储部件以外，在 CPU、设备控制器中也有存放数据的缓冲存储部件，如CPU中的通用寄存器、设备控制器中的数据缓冲寄存器等。

25.8.4上传版
b站：里昂408
公众号：里昂学长的小伙伴们